

01-vue背景介绍+声明式渲染+数据响应式

Vue3 : 写代码很爽的体验

一、背景介绍

1. 为什么要学习Vue

1.1 为什么学

1.前后端开发就业必备技能

2.岗位多，绝大互联网公司都在使用Vue，还可以助力SpringBoot、C++等项目开发

招聘中

前端开发工程师 13-20K-14薪

西安 1-3年 本科

感兴趣 立即沟通

职位描述

HTML5 Vue 计算机/软件工程相关专业 前端开发经

1、熟悉HTML、CSS等前端语言，熟练使用ES6、HTML5；

2、熟悉浏览器原理及常用网络协议，能够对项目中的安全性和

3、熟练掌握至少一个前端主流框架，熟悉VUE优先；

任职要求：

1、精通HTML5/ES6/CSS3等Web开发技能；

2、精通Angular、Vue.js、React其中的1-2种技术

3、精通sass、less高级CSS语言；

4、精通JavaScript、Ajax、DOM等前端技术，对js

5、熟练使用Ant Design、elementUI、echarts、a

6、熟悉Webpack技术；

7、熟练处理浏览器兼容性问题及调试工具；

8、有基本的Web界面的审美观，无需美工也能独

前端开发工程师（西安） 11-15K-14薪

西安 1-3年 本科

感兴趣 立即沟通

职位描述

React Vue

工作职责：

1.负责公司业务系统前端开发实现；

2.准确理解把握业务逻辑、产品需求，与后端开发协作，高效高质量项目需求；

3.产品易用性改进、前端技术优化；

4.前沿技术研究和新技术调研；

任职要求：

1.全日制统招本科以上学历，有一定的前端开发经验，扎实的前端基础，熟练掌握 HTML、

2.熟练掌握至少一种主流前端框架，例如 React、Vue.js 或其他 MVVM 框架；

3.熟练使用ES6+进行开发；熟练使用 Webpack、Fis 等构建、工程化工具；

4.关注前端主流库和框架，理解框架的设计思想及实现原理；关注业界最新技术动态和发

任先生

华为 · 人事招聘专员

3.提高开发效率

更少的时间,干更多的活,提高项目开发速度



原生js



jQuery



Vue

根据数组生成 li

1. 原生JS做法

```
<script>
  const arr = ['HTML', 'CSS', 'JavaScript', 'Vue', 'SpringBoot']
  const ul = document.getElementById('ul')
  for (let i = 0; i < arr.length; i++) {
    const li = document.createElement('li')
    li.innerText = arr[i]
    ul.appendChild(li)
  }
</script>
```

2. Vue做法

```
<ul>
  <li v-for="item in arr">{{ item }}</li>
</ul>
<script setup>
  const arr = Vue.ref(['HTML', 'CSS', 'JavaScript', 'Vue', 'SpringBoot'])
</script>
```

1.2 总结

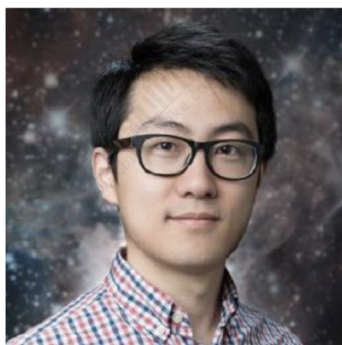
1. 为什么要学Vue?

答: 赋能、提升就业竞争力、做出更优质的项目

2. 什么是Vue

概念: Vue (读音 /vju:/, 类似于 view) 是一套 **构建用户界面** 的 **渐进式 JavaScript 框架**

Vue3官网: <https://cn.vuejs.org/>



2.1 什么是构建用户界面

基于数据渲染出用户可以看到的界面

```
title: "清仓大促",
products: [
  { id: 1, name: "手机", price: "1999元" },
  { id: 2, name: "平板", price: "2999元" },
  { id: 3, name: "电脑", price: "3999元" },
]
```

渲染结构

清仓大促

- 手机 - 1999元
- 平板 - 2999元
- 电脑 - 3999元

用户页面 (视图)

样式修饰



2.2 什么是渐进式

所谓渐进式就是 **循序渐进的学习**，不必把Vue中的所有API都学完才能开发Vue，可以学一点开发一点

Vue的两种开发方式：

1. Vue核心包开发

场景：局部模块改造

2. Vue核心包&Vue相关的帮手&工程化

场景：整站开发

2.3 什么是框架

1. 框架指的是程序员必须遵守的规则或约束

例如：安全驾驶汽车的前提条件,是理解和遵守交通规则

2. 学习Vue的过程,就是学习语法(规则)、运用规则的过程

3. 当我们掌握了Vue的使用规则了，也就学会了Vue这门框架了

具体有哪些规则, 参考 [Vue官方文档](#)

2.4 总结

1. Vue是什么？

答: 构建用户界面的 **渐进式 JavaScript框架**

2. 什么是构建用户界面？

答: 根据数据渲染生成用户界面(UI)

3. 什么是渐进式？

答: 循序渐进的学习, **学多少,用多少**

4. 什么是框架？

答: 程序员要遵守的 **语法和规则**

3. Vue学习内容和方法

3.1 思考

1. Vue学习哪些内容？

2. 学习Vue有没有事半功倍的方法？

3.2 Vue学习内容

- Vue介绍
- 数据渲染: 插值+指令+计算属性+侦听器...
- 组件及组件化: 生命周期+组件+组件化+组件通信+插槽...

- 路由: 配置+跳转+传参+嵌套+守卫...
- 状态管理: Pinia
- ElementPlus: 核心组件Form、Table、Dialog、Menu...
- 阶段实战: Vue3全家桶+ElmentPlus

3.3 Vue如何学

- 1、熟练掌握Vue语法,陌生的地方多敲几遍、尽量不抄袭代码
- 2、案例、练习、作业、项目等反复磨练,让自己主动搞起来、撸起来
- 4、多练多查多总结(Xmind等)

3.4 总结

1. 我们如何学好Vue?

答: 不抄代码, 多练多查多总结

4. 准备Vue代码编写环境

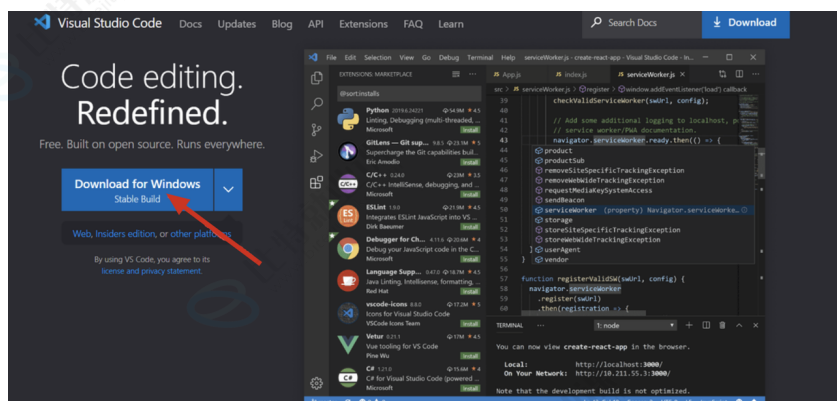
4.1 思考

1. 准备什么代码环境
2. 在哪里写Vue代码呢?

4.2 安装编辑器和插件

4.2.1 安装编辑器VSCode

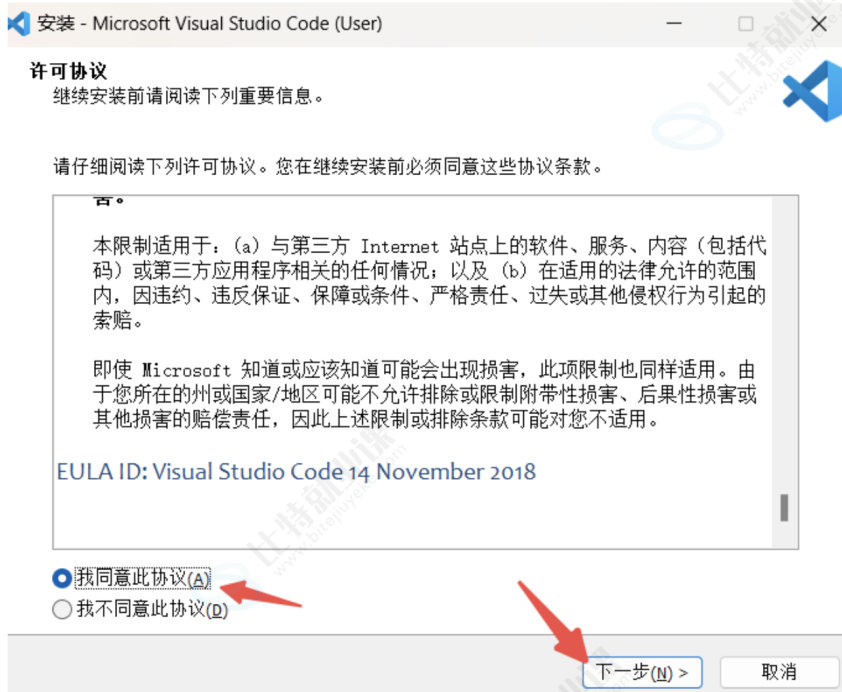
4.2.1.1 下载



4.2.1.2 双击安装

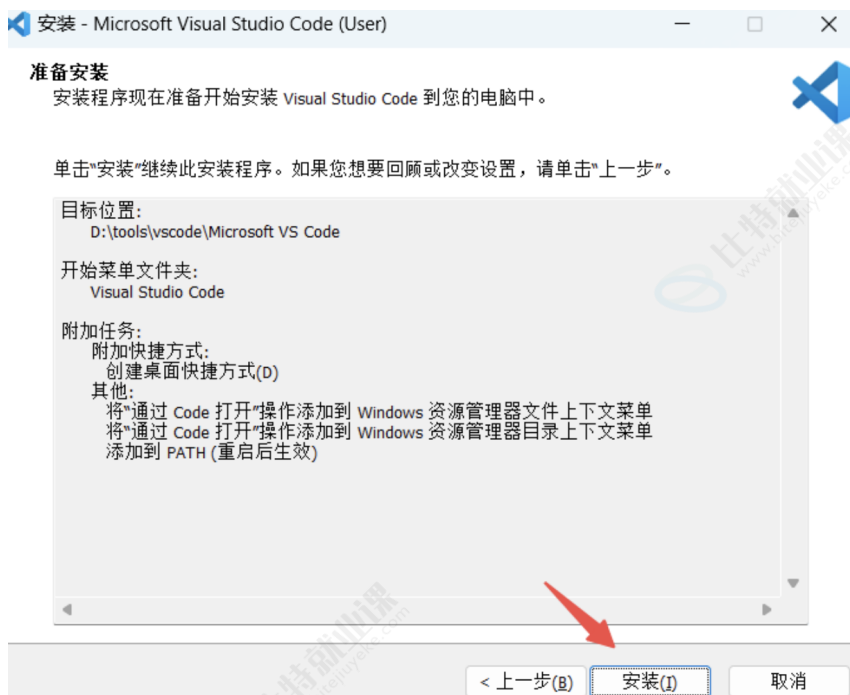
名称	类型	大小
VSCodeUserSetup-x64-1.89.1	应用程序	97.226 KB
Microsoft VS Code	文件夹	

4.2.1.3 开始安装



4.2.1.4 点击下一步、安装即可





4.2.2 必备插件

1. Vue-Official

- 提供 Vue 文件的语法高亮
- 支持 Vue 文件的智能感知和自动完成
- 提供了 Vue 文件的格式化工具

2. Vue 3 Snippets

- 提供 Vue 3 相关的代码片段，方便快速输入常见代码结构

3. Path Intellisense

- 路径自动补全

4. Auto Import

- 自动导入插件，可帮助自动完成和自动导入模块

5. Auto Close Tag

- 自动闭合HTML标签

6. Auto Rename Tag

- 自动重命名HTML标签。

4.3 Vue代码写哪

- 传统开发模式：基于html文件开发Vue,类似jQuery的使用, `<script src="vue.js">`
`</script>`

✓ VUE-TRADITION-WAY

index.css

<> index.html

JS index.js

- 工程化 开发方式: 在 构建工具(Vite/Webpack 等)环境下开发Vue, 这是最推荐的、也是企业采用的方式

✓ VUE-ENGINEERING-WAY

> node_modules

> public

✓ src

> assets

> components

▼ App.vue

JS main.js

style.css

📄 .gitignore

<> index.html

{ } package.json

📖 README.md

JS vite.config.js

📦 yarn.lock

4.4 总结

1. Vue代码有哪两种开发方式?

- 答: 1. 传统开发方式, 直接用script引入vue.js文件
2. 在工程化环境下, 借助Vite对Vue代码进行打包

二、声明式渲染

1. Vue的 Hello World 程序

1.1 CDN 引入 vuejs

```
1 <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
```

1.2 准备容器

```
1 <div id="app">
2
3 </div>
```

1.3 创建应用

```
1 <script>
2   // 创建Vue应用
3   Vue.createApp({
4     // 入口函数
5     setup() {
6       // 准备数据
7       const msg = 'Hello World'
8       // 返回数据对象
9       return {
10         msg
11       }
12     }
13   })
14   .mount('#app') // 内容渲染到 id=app 的盒子里
15 </script>
```

1.4 渲染数据

```
1 <div id="app">
2   <h1>{{ msg }}</h1>
3 </div>
```

2. setup 函数

2.1 介绍

setup函数 vue3 API的入口函数

- `setup` 函数是 `Vue3` 特有的选项，作为编写代码的起点
- 标签中用到的数据或函数，需要在 `setup` 中声明并返回

- 函数中 `this` 不是Vue实例，`setup` 中是不会用到 `this` 的

2.2 总结

1. `setup` 函数的作用

答：是Vue3代码的入口/起点、后续Vue3逻辑代码都是在这里编写的

3. 插值表达式

3.1 目标

- 掌握插值表达式的作用及语法
- 刚刚我们通过 `{{ msg }}` 渲染出 `Hello World`，在Vue3中，`{{ 表达式 }}` 称之为插值表达式



3.2 作用

把 `表达式` 的结果 展示在 `双` 标签中

3.3 语法

`{{ 表达式 }}`，表达式：凡是有结果的操作/运算，前端中常见的表达式有：

- 所有的字面量
- 对象.属性名
- 算术运算或三元运算
- 方法的调用

注意：表达式不是语句，

3.4 步骤

- `script`下的 `setup`中定义并返回初始数据
- `template`中 `插值` 展示数据

3.5 代码示例

```
1 <div id="app">
2   <!-- 1. 直接放变量 -->
3   <h1>{{ msg }}</h1>
4   <!-- 2. 对象.属性 -->
```

```

5      <p>我叫 {{ obj.name }}, 今年 {{ obj.age }} 岁</p>
6      <!-- 3. 三元表达式 -->
7      <p>{{ obj.age >= 18 ? '已成年' : '未成年' }}</p>
8      <!-- 3. 算数运算 -->
9      <p>来年我就 {{ obj.age + 1 }} 岁了</p>
10     <!-- 4. 函数的调用 -->
11     <p>fn的返回值是: {{ fn() }}</p>
12     <!-- 4. 方法的调用 -->
13     <p>{{ msg }} 中含有 {{ msg.split(' ').length }} 个单词</p>
14 </div>
15
16 <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
17
18 <script>
19   Vue.createApp({
20     setup() {
21       // 字符串
22       const msg = 'Hello World'
23
24       // 对象
25       const obj = {
26         name: '小vue',
27         age: 9
28       }
29
30       // 函数
31       function fn() {
32         return 100
33       }
34
35       // 返回
36       return {
37         msg,
38         obj,
39         fn
40       }
41     }
42   }).mount('#app')
43 </script>

```

3.6 总结

1. 什么是插值表达式?

答: `{{ 表达式 }}`, 把表达式的结果展示在 `双` 标签中

2. Vue3中的初始数据在哪声明?

答: `setup` 函数中 定义并返回 初始数据

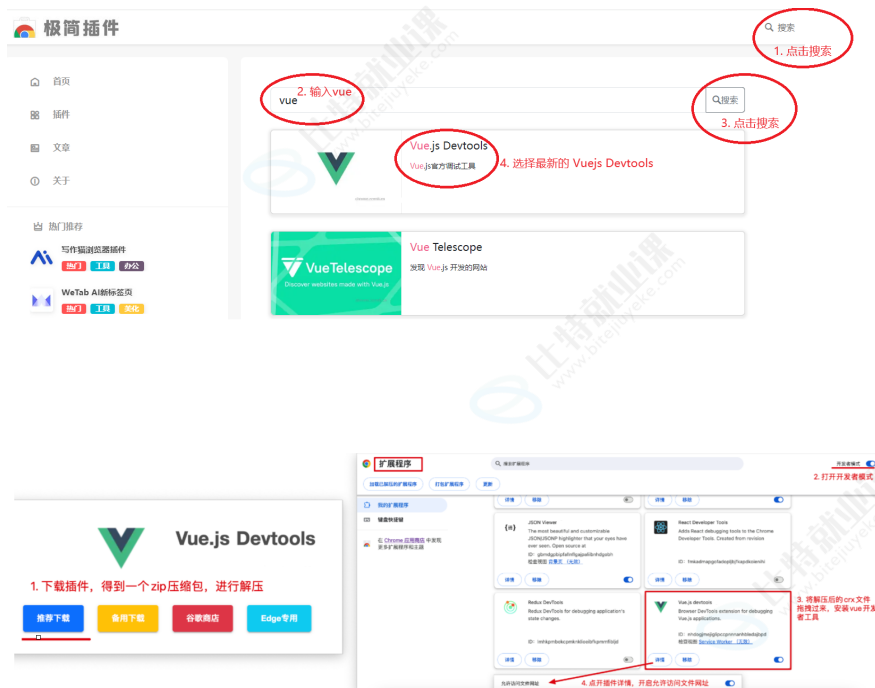
三、数据响应式

1. 安装Vue开发者工具

1.1 安装

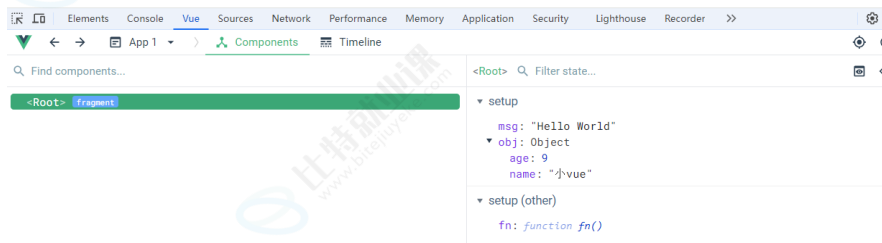
1. 通过谷歌应用商店安装（科学上网）
2. 极简插件下载（推荐） <https://chrome.zzzmh.cn/index>

安装步骤：



1.2 检测

安装之后，Chrome进入开发者工具后, 会看到多一个Vue的调试面板



2. 数据响应式

2.1 什么是数据响应式

简单的说：就是 数据变了，视图跟着变

借助开发者工具，发现数据不可改，也就是说，默认声明的数据，不具备响应式特性；换句话说，即使数据变了，视图也不会更新。

2.2 解决方案

那如何才能实现数据响应式呢？

需要借助我们接下来学习的 `reactive`、`ref` 响应式函数

3. 响应式函数 reactive

3.1 目标

掌握 `reactive` 函数定义对象 响应式数据

3.2 思考

经过之前的学习，`setup` 中默认返回数据具有响应式特性吗？

- 不是，需要使用 `reactive` 赋予数据响应式特性

3.3 使用步骤

- 从 `Vue` 中解构出 `reactive` 函数
- 在 `setup` 函数中，使用 `reactive` 函数，传入一个 普通对象，返回一个响应式数据对象
- 最后 `setup` 函数返回一个响应式对象，供模板使用

注意：`reactive`接收的 参数是对象

3.4 代码示例

```
1  <div id="app">
2    <p>我叫 {{ obj.name }}, 今年 {{ obj.age }} 岁</p>
3  </div>
4
5  <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
6
7  <script>
8    // 解构出 reactive 响应式函数
9    const { createApp, reactive } = Vue
10
11    createApp({
12      setup() {
13        // 接收一个对象作为参数
14        // 返回一个响应式对象(当对象的属性值变了，视图会跟着变)
15        const obj = reactive({
16          name: '小vue',
17          age: 9
18        })
19
20        return {
```

```
21         obj
22     }
23 }
24 }).mount('#app')
25 </script>
```

3.5 总结

1. reactive函数的作用是？

答：让一个对象具备响应式特性

2. reactive函数能转换简单类型吗？

答：不能、reactive 的参数只能是对象

4. 响应式函数ref

4.1 目标

通常使用 `ref` 函数定义响应式数据，不限类型；但如果是对象，推荐使用 `reactive`

4.2 使用步骤

- 从 `Vue` 中解构出 `ref` 函数
- 给 `ref` 传入指定类型的数据（简单or复杂均可），得到一个响应式数据

注意：操作 `ref` 创建的数据，`js` 中需要 `.value`，`template` 不可 `.value`

4.3 代码示例

```
1 <div id="app">
2   <!-- ref数据: 在 template 中使用不用 .value -->
3   <h3>{{ msg }}</h3>
4   <p>我叫 {{ obj.name }}，今年 {{ obj.age }} 岁</p>
5
6   <button @click="onClick">修改ref数据</button>
7 </div>
8
9 <script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
10
11 <script>
12   // 解构出 createApp, ref 函数
13   const { createApp, ref } = Vue
14
15   createApp({
16     setup () {
17       // ref 定义一个响应式字符串
```

```

18     const msg = ref('Hello Vue3')
19
20     // ref 定义一个响应式对象
21     const obj = ref({
22       name: '小vue',
23       age: 9
24     })
25     // 点击事件
26     const onClick = () => {
27       // 修改 msg
28       msg.value = '你好, Vue3'
29
30       // 修改 obj 的 name 和 age 属性
31       obj.value.name = 'Vue3'
32       obj.value.age = 10
33     }
34
35     return {
36       msg,
37       obj,
38       onClick
39     }
40   }
41   }).mount('#app')
42 </script>

```

4.4 总结

1. ref 函数的作用是？

答：1、把简单数据或复杂数据转换成响应式数据

2、注意：js中操作要 `.value`，模板中不可 `.value`

抛出疑问：定义响应式数据是选择 `ref` 还是 `reactive`？

5. reactive 与 ref 的选择

5.1 目标

在定义响应式数据的时候，知道是选择 `reactive`，还是 `ref`

5.2 分析

- `reactive` 只接收对象作为参数，不支持简单数据类型。
- `ref` 既可以接收基本类型，也可以接收引用类型，但在操作的时候需要 `.value`。
- 它们各有特点，现在也没有最佳实践，没有明显的界限，所有小伙伴们可以自由选择

5.3 推荐用法

- 如果数据是对象、并且字段名称也确定，推荐使用 `reactive` 赋予数据响应式，其他一律使用 `ref`

5.4 代码示例

```
1 // 1. 明确登录表单有2个字段
2 const loginForm = reactive({
3   username: '',
4   password: ''
5 })
6
7 // 2. 后台返回的数据对象，字段不确定
8 const data = ref(null)
9 const res = await axios.get('/user/10086')
10 data.value = res.data
```

5.5 总结

- 把数据变成响应式，是选择 `reactive` 还是 `ref`？

答: 1、确定字段的对象使用 `reactive`，不用 `.value`

2、其他一律使用 `ref`，并且js操作要 `.value`